Wyatt Avilla

(408) 506-2189 | wyatt@linux.com | github.com/wyatt-avilla | linkedin.com/in/wyatt-avilla | wyattavilla.dev

EDUCATION

San José State University *Aug 2025 - May 2027*
Major: M.S Software Engineering, specializing in networking software **CGPA: 4.0** Transcript
Relevant Courses: Database Systems, Operating System Design, Computer Network Design, Software Systems Engineering, Enterprise Software Platforms

University of California, Santa Cruz *Sept 2021 - June 2025*
Major: B.S. Cognitive Science, specializing in AI & HCI **CGPA: 3.9** Transcript
Minor: Computer Science
Relevant Courses: Data Structures & Algorithms, Object Oriented Programming, Parallel Programming, Computer System Design, Artificial Intelligence

SKILLS

Programming Languages: Python, Rust, TypeScript, SQL, C, C++, Nix, Shell
Backend & Data: FastAPI, REST, WebSockets, PostgreSQL, SQLAlchemy, Alembic, Tokio, Redis
Cloud & Platform: Kubernetes, Helm, Kyverno, AWS CDK, Lambda, API Gateway, DynamoDB, SNS, CloudFront, NixOS, Docker
Tooling & Systems: GitHub Actions, CI/CD, Linux, systemd, nginx, Prometheus, Concurrency, Networking

WORK EXPERIENCE

Member of the Technical Staff Oct 2025 - Present
Circuit Breaker Labs

- Built Circuit Breaker Labs' FastAPI evaluation platform for AI safety red-teaming, owning typed REST and WebSocket endpoints for single-turn and multi-turn LLM evaluation workflows
- Designed WebSocket evaluation flows with typed protocol envelopes, protocol-version validation, progress notifications, completion-request routing, and close-code error mapping, with database-backed API key authentication and monthly quota enforcement shared across REST and WebSocket handlers
- Implemented model-provider call tracking across OpenAI, OpenRouter, and WebSocket client providers, recording prompts, token counts, and errors to calculate API expenditure and preserve evaluation provenance
- Designed and migrated PostgreSQL schemas for users, API keys, test cases, generation records, test results, provider-call logs, and quotas using SQLAlchemy async and Alembic
- Packaged and deployed the API with Nix flakes, uv2nix, a NixOS service module, systemd, PostgreSQL, agenix-managed secrets, nginx TLS/WebSocket proxying, and Prometheus/Alertmanager alerting, reducing build size from 17GB to 1GB and build time from 5+ hours to under a minute by removing CUDA dependencies and pinning Nix inputs
- Refactored evaluation and provider functions to errors-as-values for per-test-case failures and diagnosed production request hangs caused by HTTP client timeouts and leaked resources after extended uptime
- Established CI/CD across Python and Nix codebases with GitHub Actions workflows for Ruff, strict Mypy, pytest coverage, and Nix builds

Software Engineer Intern May 2026 - Aug 2026
Principal Financial Group

- Extended TypeScript Helm chart-sync automation into a general transformation subsystem that parsed upstream charts, rendered templates, and patched every container definition to drop `CAP_SYS_ADMIN`
- Iteratively deployed transformed charts and triaged live Kyverno admission-controller policy reports, bringing all 51 development Kubernetes resources into compliance
- Authored and reviewed a 6,000+-line downstream pull request demonstrating the maintenance cost of injecting the security patch, then recommended that upstream chart maintainers implement the production fix
- Shipped chart-sync automation that created a Jira ticket for every generated pull request and cross-linked the two records, closing a manual release-tracking gap
- Owned AWS infrastructure for a five-person finalist intern hackathon team, provisioning API Gateway, Lambda, Bedrock, and DynamoDB with CDK for a context-aware prompt-refinement VS Code extension
- Built CI checks for testing, linting, formatting, and infrastructure validation for the completed extension, which was prepared for Visual Studio Marketplace publication
- Deployed CDK-managed SNS alerting for production CloudFront errors at a three-event/5-minute threshold and migrated two CDK applications to the enterprise pipeline while replacing deprecated GitHub Actions steps
- Diagnosed GitHub deployment failures to a repository merge-strategy misconfiguration, corrected it, and validated the fix across two core repositories
- Integrated a session-analytics platform into a Next.js-based internal sandbox to validate customer-site tracking and performed QA on the team's no-code A/B testing platform

Backend Developer Intern Sept 2024 - Dec 2024
Lillup

- Prototyped a stateless FastAPI backend integrating LangChain with a self-hosted Llama model, designing approximately 10 tools with typed JSON parsing for multi-turn interaction with structured user-profile data
- Extended Markdown parsing with markdown-it-py and regex to encode tags, progress indicators, and due dates as structured LLM
- Established backend quality gates with GitHub Actions, pytest, Mypy, and Ruff for a proof-of-concept delivered to company leadership

PROJECTS

Circuit Breaker Labs CLI January 2026 - Present
Built and shipped `cbl`, the public Rust CLI for Circuit Breaker Labs' AI safety platform, connecting to a deployed FastAPI service over WebSockets with typed protocol envelopes, API-key authentication, and version negotiation. Implemented an async engine using `tokio::select!` and `JoinSet`, with model providers abstracted behind a shared trait supporting OpenAI, Ollama, and Rhai-scripted integrations. Shipped Ratatui progress displays, 79 tests, and cargo-dist releases targeting Apple Silicon, Intel macOS, Linux musl, and Windows MSVC.

Type-Safe REST API with ESP32 Client Integration June 2025
Built a Rust REST API with Actix Web for monitoring systemd services, sharing types with ESP32 firmware for compile-time-safe client/server communication. Implemented asynchronous message passing with Embassy for low power consumption and packaged the server as a configurable NixOS service with a real-time monitoring CLI.

Open Source Contributor for Assembly Reverse Engineering January 2024 - April 2024
Contributed 10 pull requests translating 2,800 lines of PowerPC assembly into 3,200 lines of C for *Super Smash Bros. Melee*. Ensured byte-perfect accuracy through GitHub Actions CI that validated the compiled binary against the original and collaborated through code reviews with a distributed team.